

Data Representations

Peter Sørensen

Table of contents

1	Individual-Level PLINK BED Representation	1
2	Dense Summary-Statistic Representation	2
3	Disk-Backed CSR Sparse-LD Representation	3
4	Sufficient Statistics: wy, ww, and yy	4
5	Multi-Trait Cross-Products, Sample Overlap, and SSY	5
6	Marker Ordering and Annotation Alignment	6
7	Representation-Specific Limitations and Validation Checks	6
8	Related Notes	7

`sblr` can fit Bayesian linear regression models from two broad forms of genomic data:

1. **individual-level data**, where marker genotypes and phenotypes are available for the same individuals; and
2. **summary statistics plus linkage disequilibrium (LD)**, where the sampler receives cross-products that summarize the genotype-phenotype data rather than the full observed matrices.

These are two representations of closely related regression calculations, not two unrelated model families. The representation determines what must be stored, what can be recomputed during sampling, and which alignment and sample-overlap assumptions must be made explicit.

1 Individual-Level PLINK BED Representation

Let X be an $n \times m$ genotype matrix for n individuals and m markers, and let Y be an $n \times n_t$ phenotype matrix for n_t traits. For one trait, Y reduces to a vector y . A Bayesian linear regression model can be written as

$$Y = XB + E,$$

where B contains marker effects and E contains residuals. In a single-trait model, this is $y = Xb + e$.

With individual-level data, a marker update can read the observed genotype column x_j and evaluate quantities such as

$$x_j^\top x_j \quad \text{and} \quad x_j^\top e = x_j^\top (y - Xb).$$

The sampler can therefore update effects from observed marker values and current individual-level residuals. It does not need a separately supplied LD matrix because correlations among markers are implicit in X .

PLINK BED files store genotype calls compactly on disk rather than as an in-memory numeric matrix. In the package workflow, a qgg `Glist` supplies the BED file paths and related metadata:

- `Glist$bedfiles` identifies the BED files;
- `Glist$n` gives the number of individuals;
- `Glist$rsids` identifies the marker columns in each BED file;
- `Glist$aaf` or `Glist$maf` supplies allele frequencies or MAF used when markers are standardized or when MAF-dependent priors are used;
- `Glist$rsidsLD` identifies the selected markers and their intended LD order.

`stblr_bed()` is the high-level direct BED interface. It receives the `Glist`, phenotype vector or matrix y , optional chromosome selection, and optional marker column indices `cls`. Its BED samplers read marker blocks from disk and perform marker updates without first materializing all of X or a full $X^\top X$ matrix. Optional `rows` select 1-based individual indices, and `scale = TRUE` standardizes markers using the corresponding allele frequencies. `stblr_bed_marker()` remains available as a lower-level BED marker interface.

The individual-level representation is information-rich: if the same individuals have multiple measured traits, cross-trait phenotype and residual cross-products can be calculated directly. Its main costs are access to sensitive individual-level data and repeated genotype reading during model fitting.

2 Dense Summary-Statistic Representation

The Gaussian linear model likelihood and marker-update calculations can be expressed using cross-products. For one trait, the central quantities are

$$wy = X^\top y, \quad XX = X^\top X, \quad yy = y^\top y,$$

together with the sample size n . For multiple traits,

$$WY = X^\top Y$$

contains one marker-trait cross-product column per trait.

The matrix $X^\top X$ describes both marker sums of squares on its diagonal and marker-marker LD in its off-diagonal entries. In package interfaces, `ww` denotes per-trait marker sums of squares or corresponding diagonal information, while `LD` supplies the off-diagonal relationships needed to propagate an effect update to correlated markers. Depending on scaling and the producing function, `LD` may be represented in correlation units or as the corresponding genotype cross-products. The statistics and `LD` representation must use the same marker scaling and order.

These quantities are sufficient for many marker updates because the current marker-residual score can be represented as

$$r = X^\top(y - Xb) = wy - X^\top Xb.$$

If marker j 's effect changes by Δb_j , the updated score can be calculated from column j of $X^\top X$:

$$r_{\text{new}} = r_{\text{old}} - (X^\top X)_{.j} \Delta b_j.$$

This is why a summary-statistics sampler needs more than marginal genotype-phenotype associations. It needs enough information to reproduce the cross-products used by the marker-update and variance calculations. Marginal `wy` values without compatible `LD` cannot account for correlated markers, and `LD` without `wy`, `yy`, and n does not define the phenotype side of the likelihood.

A dense representation stores all of $X^\top X$, which requires $O(m^2)$ entries. That becomes impractical for large marker sets. The disk-backed sparse-`LD` representation retains the same conceptual role while dropping small or distant marker-marker entries.

3 Disk-Backed CSR Sparse-LD Representation

Compressed sparse row (CSR) format stores only retained entries of a sparse matrix. Conceptually, it records:

- a vector of row pointers delimiting each matrix row;
- column indices for retained entries; and
- the retained `LD` or cross-product values.

`sparseLD_stream_CSR()` computes sparse `LD` directly from one or more PLINK BED files and writes a disk-backed CSR representation at `out_prefix`. The selected marker columns are supplied as a list through `cls`, one vector per BED file. Optional `rows` and `af` must describe the same individuals and marker scaling used for the sufficient statistics.

Sparsification is controlled by:

- `r2_threshold`, which removes pairs below a squared-correlation threshold;
- `max_distance_bp`, which can restrict pairs by base-pair distance;
- `max_distance_variants`, which can restrict pairs by marker-index distance;
- `block_size` and `nthreads`, which control construction rather than the statistical definition.

The sparse representation approximates $X^\top X$. It reduces memory and disk requirements when most long-range or weak marker relationships can be ignored, but the threshold and distance rules become part of the analysis. Two fits using different sparse-LD construction rules do not have identical inputs even if they began from the same BED file.

`sparseLD_read_CSR()` reads files written by `sparseLD_stream_CSR()` and returns row pointers, column indices, and values. Its `one_based` option controls the indexing convention of returned column indices. This reader is useful for checking dimensions, indexing, and retained entries; `stblr_csr()` uses the disk-backed prefix directly.

Detailed storage and construction contracts belong in [Technical sparse LD and CSR](#).

4 Sufficient Statistics: `wy`, `ww`, and `yy`

`make_stats()` is the package bridge from a BED-backed `Glist` and phenotypes to the sufficient statistics consumed by `stblr_csr()`. It resolves BED files, selected chromosomes, selected BED marker columns `cls`, allele frequencies, phenotype-to-BED row matching, and marker names from `Glist`. Optional `rows` and `scale` determine which individuals and marker standardization are used. The lower-level `bed_xtx_xty()` helper performs per-BED cross-product work.

The returned `stats` object contains the marker-trait cross-products and related quantities required by the ST-BLR wrapper:

- `stats$wy`: $X^\top Y$, represented by trait;
- `stats$ww`: marker sums of squares or diagonal genotype cross-product information, represented by trait;
- `stats$yy`: $y_t^\top y_t$ for each trait;
- `stats$n` and `stats$m`, when available, for sample and marker counts.

`make_sparseLD()` computes disk-backed sparse LD from the same BED-backed `Glist` and stores the resulting CSR prefix and resolved marker metadata in `Glist$sparseLD`. This metadata includes the sparse-LD prefix, marker order, selected BED columns, rows, allele frequencies, distance limits, threshold, and construction settings used by CSR models. `stblr_csr()` combines `stats` with this sparse-LD metadata from `Glist`, or with an explicit `ld_prefix` when supplied. The wrapper initializes per-trait variance quantities using `stats$yy`, n , and m , then passes `wy`, `ww`, `yy`, and the CSR prefix to the sparse-LD sampler.

The complete conceptual bridge is:

```

BED files + aligned phenotypes
|
+--> make_stats() -----> wy, ww, yy, n, marker names
|
+--> make_sparseLD() -----> Glist$sparseLD prefix and metadata
                                |
                                v
                                stblr_csr() summary-statistics model

```

The two branches must be produced from the same marker selection, sample selection, allele frequencies, and scaling convention. In contrast, `stblr_bed()` follows a direct branch from BED files and phenotypes to the individual-level sampler. The worked comparison is described in [Sparse-LD and BED workflows](#).

5 Multi-Trait Cross-Products, Sample Overlap, and SSY

The phrase “multi-trait summary statistics” can refer to two different levels of information.

First, traits can be fitted as separate or parallel single-trait regressions. For trait t , the phenotype sum of squares is

$$yy_t = y_t^\top y_t = \sum_i y_{it}^2.$$

A vector of `yy` values, with corresponding columns of $X^\top Y$, is enough to initialize and fit per-trait likelihood calculations that do not require observed cross-trait phenotype products. The current `stblr_csr()` wrapper uses this per-trait representation: it infers the number of traits from `length(stats$yy)` and uses each `yy_t` to initialize trait-specific variances.

Second, a genuinely joint multi-trait likelihood with a full residual covariance matrix requires the cross-trait phenotype matrix

$$\text{SSY} = Y^\top Y, \quad \text{SSY}_{ts} = \sum_{i \in \mathcal{J}_{ts}} y_{it} y_{is},$$

where $\mathcal{J}_{ts}$ is the set of individuals observed for both traits. The diagonal entries are the univariate `yy_t`; the off-diagonal entries contain information that a vector of `yy` values does not preserve.

With individual-level Y , `SSY` and the effective overlap counts can be calculated from observed individuals. With summary statistics, they must be provided or reconstructed. If N_{ts} is the number of individuals shared by traits t and s , and R_y is a phenotypic correlation matrix, a common approximation is

$$\widehat{\text{SSY}} = N \odot \widehat{\text{Cov}}(Y),$$

with diagonals forced to the observed `yy_t`. This approximation assumes that overlap counts and phenotypic covariance information are meaningful for the study design. Full overlap, partial overlap, missingness, and meta-analysis can imply materially different N matrices.

Without cross-trait `SSY` and overlap information, a summary-statistics model cannot reconstruct the cross-products needed for full residual covariance updates. The complete derivation and assumptions are preserved in [Technical multi-trait sample overlap](#).

6 Marker Ordering and Annotation Alignment

Marker order is a correctness condition, not presentation metadata. Every marker-indexed object must refer to the same biological marker at position j :

```
BED marker column selected by cls
  = Glist$rsidsLD marker j
  = sufficient-statistic row j
  = sparse-LD row and column j
  = annotation-matrix row j
  = posterior-summary row j
```

In the workflows, the intended LD marker order is established by `Glist$rsidsLD[[chr]]`. The BED column indices are derived with

```
cls <- match(Glist$rsidsLD[[chr]], Glist$rsids[[chr]])
stopifnot(!anyNA(cls))
```

The same `cls` is then supplied to `make_stats()` and `make_sparseLD()`. Allele frequencies or MAF are subset with the same indices:

```
maf_ld <- Glist$maf[[chr]][match(Glist$rsidsLD[[chr]], Glist$rsids[[chr]])]
```

Do not use `%in%` for this alignment because it tests membership but does not preserve the requested LD/model order. For annotation models, the $m \times K$ annotation matrix **A** must have one row per selected marker in exactly this order. The annotation workflow makes that contract explicit by assigning `Glist$rsidsLD[[chr]]` as row names and checking that they are identical to the expected marker IDs.

Posterior matrices such as `fit$bm` and `fit$dm` inherit the marker ordering used by the model input. A plausible-looking result is still invalid if its rows are joined to annotations or marker metadata in a different order.

Sample alignment is equally important. When `rows` is used, the phenotype rows and every BED-derived object must describe the same individuals. Sufficient statistics generated from one sample subset cannot be combined with sparse LD generated from another without changing the implied model.

7 Representation-Specific Limitations and Validation Checks

Individual-level and summary-statistics representations have different failure modes.

Individual-level BED fitting

- requires access to individual genotypes and aligned phenotypes;
- reads genotype blocks repeatedly during fitting;

- can calculate observed phenotype and residual cross-products when data are jointly available;
- still requires correct `cls`, `rows`, allele frequencies, and scaling.

Summary statistics plus sparse LD

- can avoid repeated access to individual-level genotypes after preparation;
- replaces the full $X^T X$ matrix with an explicit LD approximation;
- requires compatible `wy`, `ww`, `yy`, `n`, and LD;
- requires `SSY` and overlap information for full joint multi-trait residual covariance calculations;
- cannot recover information omitted during summarization or sparsification.

At minimum, validate the following before fitting:

1. `cls` contains no missing marker matches and has the expected length.
2. Allele frequencies correspond to the selected marker columns.
3. `make_stats()` and `make_sparseLD()` use identical marker order, sample rows, and scaling choices.
4. CSR dimensions and indexing are consistent with the number of markers.
5. Annotation rows and marker/group vectors use the same marker order.
6. `stats$wy`, `stats$ww`, and posterior marker matrices have compatible dimensions.
7. Trait names, `stats$yy`, and phenotype columns are aligned.
8. Multi-trait summary analyses state how `SSY` and sample overlap were observed, assumed, or reconstructed.

Comparing posterior summaries from `stblr_bed()` and `stblr_csr()` on the same prepared data is a useful integration check, but exact numerical agreement is not expected automatically. Sparse-LD approximation, scaling, sampler scheduling, finite MCMC error, and prior settings can all affect the comparison.

8 Related Notes

- [Sparse-LD and BED workflows](#)
- [Technical summary statistics](#)
- [Technical multi-trait sample overlap](#)
- [Technical sparse LD and CSR](#)